

SDUI 제품화·판매 전략 — 브레인스토밍 결론 & 로드맵

(2026-07-09)

출처: 2026-07-09 인터랙티브 브레인스토밍 세션. "SDUI 구조를 템플릿화해서 판매한다"는 목표에서 출발. 이 문서는 **전략 방향의 확정본**이며, 실행 계획은 뒤쪽 섹션에서 MVP 단위로 발전시킨다. 관련: [SDUI/CLAUDE.md](#) — 엔진 아키텍처, [0707_design_unification_plan.md](#)

0. 한 줄 요약

"**AI 네이티브 풀스택 SDUI 엔진**" 을 오픈코어로 풀어 개발자를 유입시키고, AI 컴포넌트·호스팅·프리미엄 템플릿을 유료화하며, 마켓플레이스로 확장한다. K-RIDE는 상품이 아니라 "**이 엔진으로 진짜 앱이 나온다**"는 플래그십 데모다.

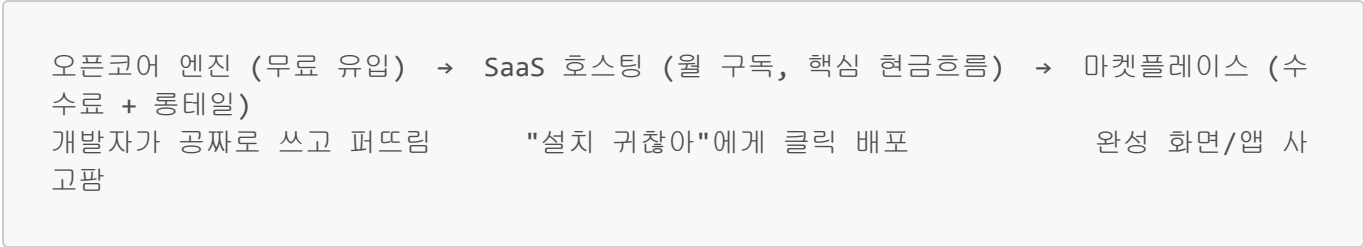
1. 팔 만한 자산 (이미 보유)

자산	근거
런타임 렌더링 엔진	<code>ui_metadata</code> 트리를 <code>componentMap</code> 으로 해석 → 클라이언트 재배포 없이 DB만 바뀌 UI 변경
메타데이터 화면 정의	<code>screen_id</code> / group 트리 / <code>repeater(ref_data_id)</code> 데이터 바인딩
쿼리 마스터	SQL을 DB(<code>query_master</code>)에 저장 → 동적 페칭, Redis 캐시
어드민 프리뷰 툴 + 테마 토큰	<code>design_tokens</code> 테이블, <code>ThemeProvider</code> , SDUI 어드민 프리뷰(#58)
AI 네이티브 컴포넌트	KRIDE 챗봇 SSE 스트리밍, AI 일정 생성 — 경쟁자 대비 핵심 차별점
풀스택 일체형	프론트 엔진 + 백엔드 + <code>query_master</code> + RBAC + 인증
검증된 실서비스	K-RIDE(여행 소비자 앱)가 프로덕션에서 실제 구동

2. 확정된 전략 결정

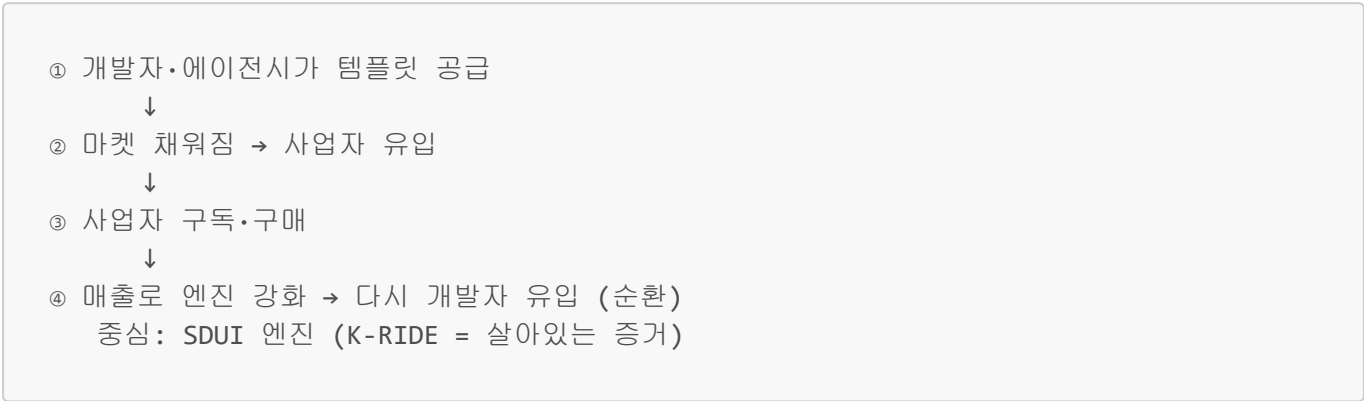
결정 항목	확정 내용
제품 형태	오픈코어 엔진 → SaaS 호스팅 → 템플릿 마켓플레이스 (3단 사다리)
고객	개발자/에이전시(공급, 먼저 공략) → 비개발 사업자(수요, 나중) — 양면 시장
포지셔닝	AI 네이티브 + 프론트·백·인증 일체형
첫 씨앗 템플릿	① 범용 스타터 킷 + ② AI 챗봇 앱 킷 (수직 앱 아님 = 수평 플랫폼)
K-RIDE 역할	상품 ❌ → 플래그십 데모/전시장 ✅

오픈코어 공식 (3단 사다리)



선례: Supabase / Strapi / WordPress.

양면 마켓플레이스 플라이휠



마켓플레이스의 "닭이 먼저냐 달걀이 먼저냐" 문제 → 공급(개발자)을 먼저 씨앗으로 깬다.

3. 경쟁 지형 & 차별점

카테고리	예시	한계
프론트 비주얼 빌더	Builder.io, Plasmic	프론트만, 백엔드·데이터·인증 없음
사내 툴	Retool, Appsmith	관리자 화면 전용, 소비자 앱 부적합
노코드 앱빌더	FlutterFlow, WeWeb, Toddle	강력하나 AI가 얄음
헤드리스 CMS	Strapi	콘텐츠만, UI 렌더링 엔진 아님

우리만의 3무기: ① AI 네이티브(챗봇·생성이 컴포넌트로 박힘) ② 풀스택 일체형 ③ 검증된 실서비스.

4. 오픈코어의 급소 — 무료/유료 경계선

이 선이 매출을 결정한다. 간판(AI 네이티브)과 유료 지점을 일치시킨다: "엔진은 공짜, AI와 편의는 유료."

● 무료 (오픈코어 · 유입)	● 유료 (SaaS + 마켓 · 매출)
렌더링 엔진 (DynamicEngine, componentMap)	AI 컴포넌트 (챗봇 SSE, AI 생성) ← 왕관 보석
기본 컴포넌트 · 메타데이터 트리	호스팅 어드민 · 프리뷰 툴
범용 스타터 키트 1종	프리미엄 템플릿 · 고급 RBAC/테마

5. MVP — 첫 3수(手)

1. 엔진 추출(carve-out) metadata-project에서 K-RIDE 종속 코드(KRIDE_*, 여행 도메인)를 걷어내고 순수 엔진 + 범용 스타터만 남긴 별도 리포로 분리. → 난이도 평가 필요: K-RIDE 결합도 스캔(아래 섹션 6).
2. AI 챗봇 컴포넌트 플러그인화 하드코딩된 KRIDE 챗봇(KrideChatComponent, useKrideChatStream)을 설정만 바꾸면 붙는 범용 AI 컴포넌트로 일반화 → 유료 라인의 핵심.
3. 랜딩 + 라이브 데모 + GitHub 오픈코어 리포 K-RIDE를 "이 엔진으로 만든 예시"로 박제한 데모 페이지 + 오픈코어 리포 공개 → 개발자 유입 시작.

6. K-RIDE 결합도 스캔 결과 (2026-07-09 완료)

전체: metadata-project 내 449회 / 59파일. 대부분 fields/kride/ 폴더에 격리됨. 진짜 코어 오염은 4개 파일에 집중.

파일	결합 내용	난이도	처리 방식
fields/kride/** (~40파일)	여행 전용 컴포넌트가 한 폴더에 격리	쉬움	통째로 "여행 템플릿 번들"로 이동
constants/screenMap.ts	KRIDE_INTRO1~5, FOCUS, CHAT 등 8개 URL 매핑	쉬움	엔트리 삭제
constants/componentMap.tsx	kride import 10 + 레지스트리 5(KRIDE_CHAT, MAP_VIEW...)	쉬움	플러그인 등록으로 이전
DynamicEngine/hook/useBaseActions.tsx	isKrideScreen() + kride_form localStorage 영속화	중간	persistFormData 설정 플래그로 일반화
app/view/[...slug]/page.tsx	KRIDE_FOCUS 챗 모달 ·itinerary 상태·KRIDE_ 분기 36 곳	높음	범용 라우터 리팩터 — 진짜 공수

종합 판단: 엔진 추출 MVP의 80%는 이동·삭제, 20%가 page.tsx 리팩터. page.tsx의 여행 특수 분기를 플러그인/설정 메커니즘으로 빼내는 게 핵심 병목이자, "순수 엔진" 컨셉을 위해 어차피 해야 하는 정리 작업.

남은 조사 항목 (미착수)

- ☐ 무료/유료 기능 게이팅 방식 (라이선스 키 / 빌드 분리 / 서버 검증).
- ☐ SaaS 멀티테넌시 설계 (screen_id 네임스페이스, DB 격리).
- ☐ 가격 모델 구체화 (시트/사용량/AI 호출 기반).
- ☐ 마켓플레이스 템플릿 패키징 포맷 (ui_metadata + query_master + 컴포넌트 번들 export/import).